

GitHub och GitHub Desktop

Samarbete, konflikter och Code Review

Mål för dagen

Efter dagens lektion ska du kunna:

- förstå hur branches används i ett grupparbete
- förstå vad som händer när två personer ändrar samma kod
- använda GitHub Desktop
- lösa en Git-konflikt i VS Code
- skapa och arbeta med GitHub Issues
- skapa en Pull Request
- granska en annan students kod
- använda tydliga commit-meddelanden
- arbeta från Issue → Branch → Pull Request → Merge

DEL 1 Repetition

1. Vad betyder de viktigaste Git-begreppen?

Begrepp	Betydelse
Repository	Hela projektet
Clone	Hämtar projektet från GitHub till datorn
Branch	En egen arbetsgren
Commit	Sparar en version av ändringarna
Push	Skickar ändringar till GitHub
Pull	Hämtar ändringar från GitHub
Pull Request	Förslag att lägga in ändringar i en annan branch
Review	En annan person granskar koden
Merge	Slår ihop ändringarna med exempelvis main

2. Det normala arbetsflödet

Ett vanligt arbetsflöde är:



Kom ihåg: Du arbetar normalt inte direkt på main när du utvecklar en ny funktion utan skapar en feature.

GitHub Desktop

Samma arbetsflöde i **GitHub Desktop**. GitHub Desktop är ett grafiskt gränssnitt för Git. Det betyder att ni fortfarande använder Git, men istället för att skriva alla kommandon kan ni klicka på knappar. Jag visar samma flöde för en branch som i förra övningen. GitHub Desktop måste installeras lokalt. Ladda ner det från [Download GitHub Desktop | GitHub Desktop](#) och logga in i internet GitHub.

1. Öppna projektet

Starta GitHub Desktop och välj projektets repository, alltså det vi jobbat med igår när ni körde individuellt. Ni bör kunna se projektet `lexicon_project`. Därefter, koppla ihop Desktop med VS Code, detta alternativ syns i fönstret.

No local changes

There are no uncommitted changes in this repository. Here are some friendly suggestions for what to do next.



Open the repository in your external editor

Select your editor in [Options](#)

Repository menu or **Ctrl + Shift + A**

Open in Visual Studio Code

När ni väljer det, öppnas VS Code automatiskt.

2. Skapa en branch i GitHub Desktop. I meny välj

Branch-new Branch

Döp det till *feature/project*. I GitHub Desktop motsvarar det kommandot i Terminalen:

```
git switch -c feature/project
```

3. Gör ändringar

I VS Code bör du nu stå i *feature/project*. Gör ändringar precis som tidigare, spelar ingen roll i vilken fil. Spara allt i VS Code. När ni sparar filerna kommer GitHub Desktop att upptäcka ändringarna.

4. Skapa commit

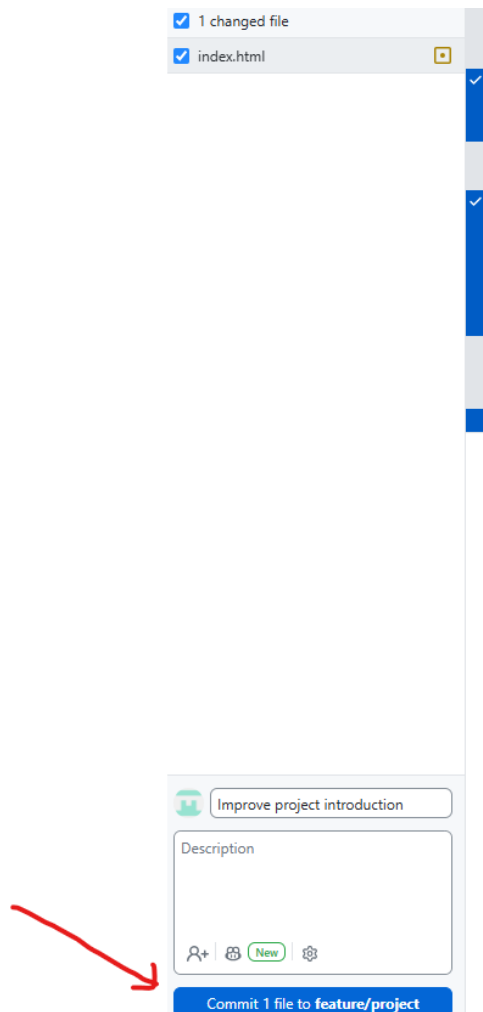
I GitHub Desktop visas nu dina ändrade filer. Skriv ett meddelande, exempelvis: Improve project introduction och klicka på:

Commit to feature/project

Det motsvarar:

```
git add .
```

```
git commit -m "Improve project introduction"
```



5. Push

Klicka: **Push origin**

Det motsvarar:

git push

GitHub Desktop skickar nu branchen till GitHub.

6. Nu kan du utföra Pull Request och merge från feature/project till main i GitHub. Precis som i övningen igår. Det går dock att utföra

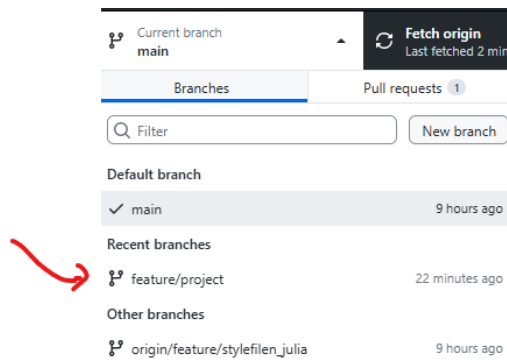
7. Pull Request

Efter push kan GitHub Desktop erbjuda att hämta det uppdaterade main från GitHub till VS Code. Då byter ni branch till main, går ni till Repository-Pull och väntar på uppdateringen.

8. Vi testar arbeta i GitHub Desktop och därför skall vi prova på att utföra push/merge direkt i det verktyge utan att behöva göra något i GitHub. Gå tillbaka till feature/project branchen i GitHub Desktop

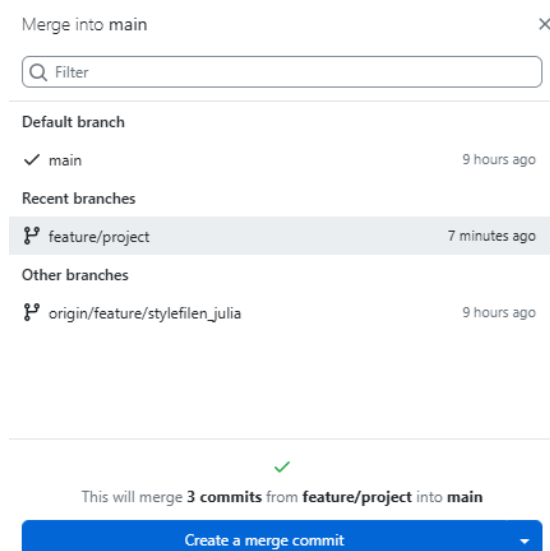
Julia Åhlén

Lexicon



9. Öppna i VS Code, det bör flyttas till feature/project automatiskt. Gör ytterligare en ändring i VS Code i en valfri fil. Spara.

10. Nu syns det i GitHub Desktop att ni genomfört ändringen. Då kan ni slå ihop den ändringen med main lokallt. Byt till main i Current Branch fliken. Välj sedan Branches-Merge to Current Branch. Välj din branch feature/project att Merge into main. Sedan Klicka Create a merge commit.



Efter det steget är main uppdaterad lokalt med det som finns i feature/project. Vi kan nu flytta hela main till GitHub med Push origin knappen.

No local changes

There are no uncommitted changes in this repository. Here are some friendly suggestions for what to do next.



Push commits to the origin remote

You have 3 local commits waiting to be pushed to GitHub.

Always available in the toolbar when there are local commits waiting to be pushed or

Ctrl + P

Push origin

Julia Åhlén
Lexicon

När det är klart, flytta till GitHub online och notera att din main är förändrad.

11. Du kan även skapa en Pull till GitHub Desktop. Dock finns det ingen poäng med detta nu, eftersom vi nyss uppdaterat main på GitHub. Annars skulle vi behöva gå till Repository-Pull.

Terminal eller GitHub Desktop? Ni har nu arbetat med Git på två olika sätt.

Terminal

```
git add .  
git commit -m "..."  
git push  
git pull
```

GitHub Desktop

Ändringar

↓

Commit eller Merge till main

↓

Push

Det är samma Git-system under ytan. Skillnaden är framför allt hur ni kommunicerar med Git.

DEL 2 Vad händer när två personer ändrar samma kod?

Detta kallas en **merge conflict** eller **Git-konflikt**.

Git kan automatiskt slå ihop många ändringar. Men ibland vet Git inte vilken ändring som ska användas. Då måste en människa bestämma.

Övning 1 Skapa en konflikt

Steg 1

Alla ska börja från den senaste versionen av main (gårdagens övning). Jobba i samma grupp som igår. Alltså alla bör ha samma GitHub repository klonad på VS Code.

Kör:

```
git switch main
```

```
git pull
```

Julia Åhlén

Lexicon

Kontrollera:

git status

Det är två personer som skall skapa ändringar på samma fil. Ni får välja vem som blir boven i dramat. Notera att jag ger instruktioner i Terminalen, dock får ni välja att arbeta via GitHub Desktop. I så fall använd Commit från GitHub Desktop och sedan flytta till GitHub för att utföra Pull Requests.

Steg 2 Student A skapar en branch

git switch -c feature/student-a

Ändra en rad i script.js.

Exempel:

```
document.querySelector("#message").textContent =  
  "Hello from Student A";
```

Gör:

Stage (git add .)

↓

Commit (git commit -m " Update message for student A ")

↓

Push (git push)

Steg 3 Student B skapar en annan branch

Student B ska också utgå från den ursprungliga main.

Skapa:

git switch -c feature/student-b

Ändra samma rad till:

```
document.querySelector("#message").textContent =  
  "Hello from Student B";
```

Gör:

Stage (git add .)

↓

Commit (git commit -m " Update message for student B ")

↓

Julia Åhlén
Lexicon
Push (git push)

DEL 3 Skapa en konflikt

Student A skapar en Pull Request:

feature/student-a → main

En annan student granskar.

Godkänn och merga Pull Requesten.

Nu finns Student A:s ändring i main.

Student B skapar Pull Request

Student B försöker nu göra:

feature/student-b → main

Eftersom Student A redan har ändrat samma rad i main, medan Student B har ändrat samma rad på sitt sätt, kan GitHub inte automatiskt slå ihop ändringarna. GitHub visar då att Pull Requesten har en konflikt, exempelvis:

This branch has conflicts that must be resolved

eller:

Merge conflict

VIKTIGT! Lös inte konflikten på GitHub.

Klicka alltså inte på: **Resolve conflicts**

Vi ska istället lösa konflikten i **VS Code**, så att vi får se hur Git hanterar en konflikt lokalt. Låt Pull Requesten vara öppen på GitHub och gå tillbaka till **VS Code**.

DEL 4 Lös konflikten i VS Code

Git kan visa något liknande i editorn när ni öppnar konfliktfilen:

```
<<<<<<< HEAD document.querySelector("#message").textContent = "Hello from  
Student B";  
=====  
document.querySelector("#message").textContent = "Hello from Student A";  
>>>>>> origin/main
```


Julia Åhlén
Lexicon

Git säger i princip: "Jag vet inte vilken version du vill behålla."

VS Code visar ofta alternativ

Du kan få alternativ som:

Accept Current Change

Behåll den aktuella versionen.

Accept Incoming Change

Behåll den andra versionen.

Accept Both Changes

Behåll båda.

I den här övningen

Bestäm tillsammans vilken text som ska användas.

Exempel:

```
document.querySelector("#message").textContent =  
    "Hello from our group!";
```

Ta bort konfliktmarkeringarna:

```
<<<<<<<  
=====  
>>>>>>>
```

Och lägg in er version.

Spara filen.

DEL 5 Commit och Push efter konflikten

När konflikten är löst:

1. Spara filen.
2. Gå till **Source Control** eller **Terminal**.
3. Kontrollera ändringen.
4. Stage filen.
5. Gör en commit. Exempel: Resolve merge conflict
6. Push.

Sedan kan Pull Requesten mergas på GitHub.

En konflikt betyder inte att Git är trasigt. Det betyder: Två personer har gjort ändringar som Git inte automatiskt kan kombinera. Det är därför code review och kommunikation är viktigt i ett grupparbete.

DEL 6 En till konflikt

Börja från samma main

Välj två personer i gruppen som skall utföra konfliktsituation.

1. Öppna GitHub Desktop.
2. Välj repositoryt.
3. Kontrollera att ni står på main.
4. Klicka **Fetch origin**.
5. Klicka **Pull origin** om den visas.

Nu ska båda ha samma utgångsläge.

Steg 2 Person A skapar en branch

Person A:

I GitHub Desktop:

Current Branch → New Branch

Namn:

feature/person-a

Skapa branchen.

Sedan öppnar Person A projektet i VS Code.

Steg 3 Person A skapar filen

Person A skapar test.txt med innehållet:

Detta är Person A:s version.

Spara filen.

I GitHub Desktop ska filen nu synas under **Changes**.

Skriv commit-meddelande: Skapa test.txt

Klicka:

Julia Åhlén
Lexicon

Commit to feature/person-a

Sedan:

Push origin

Nu finns Person A:s branch på GitHub.

Steg 4 Person B skapar en egen branch

Nu är det viktigt att Person B **inte hämtar Person A:s branch**.

Person B ska fortfarande utgå från den gamla main.

I GitHub Desktop:

Current Branch → main

Kontrollera:

main

Person B skapar sedan:

Current Branch → New Branch

Namn:

feature/person-b

Steg 5 Person B skapar samma fil

Person B öppnar projektet i VS Code och skapar också test.txt Men innehållet är annorlunda:

Detta är Person B:s version.

Spara.

I GitHub Desktop:

Commit to feature/person-b

Commit-meddelande:

Skapa test.txt

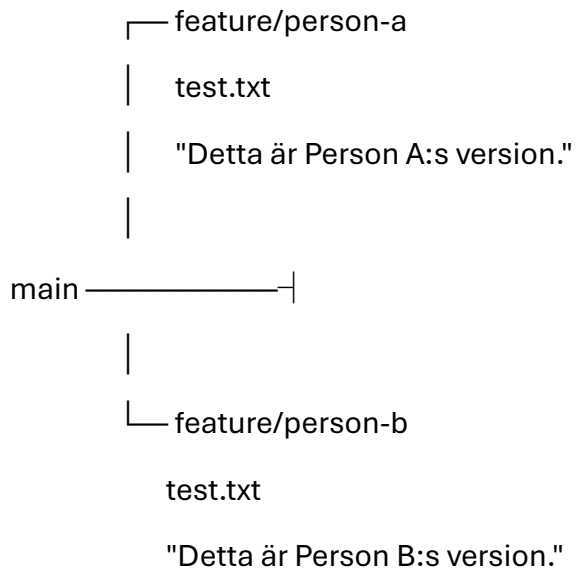
Sedan:

Push origin

Julia Åhlén
Lexicon

Nu har ni två olika versioner

På GitHub ser det ungefär ut så här:



Det viktiga är att båda brancherna skapades från samma gamla main, innan test.txt fanns.

Steg 6 Merga Person A

Nu går Person A, eller den som ansvarar för mergen, till GitHub Desktop.

Byt till:

main

Sedan:

Branch → Merge into current branch

Välj:

feature/person-a

Git kan merga detta utan problem eftersom test.txt inte finns i main.

Efter mergen har main:

test.txt

Detta är Person A:s version.

Gör **Push origin**

Nu finns Person A:s version på GitHub i main.

Julia Åhlén
Lexicon

Steg 7 Försök merge Person B

Nu kommer det intressanta. Fortfarande på:

main

Välj:

Branch → Merge into current branch

Välj:

feature/person-b

Nu kommer Git att upptäcka att båda brancherna har skapat test.txt, men med olika innehåll. Det går inte att automatiskt bestämma vilken version som är rätt.

Du får en **merge conflict**.

Steg 8 Titta på konflikten i VS Code

Öppna test.txt i VS Code. Det kommer sannolikt att se något ungefär som:

<<<<<<< HEAD

Detta är Person A:s version.

=====

Detta är Person B:s version.

>>>>>>> feature/person-b

Det betyder:

<<<<<<< HEAD

= versionen som redan finns i main.

=====

= gränsen mellan versionerna.

>>>>>>> feature/person-b

= versionen från branchen som du försöker mergea.

Steg 9 Lös konflikten

I VS Code kan du få knappar som:

- **Accept Current Change**
- **Accept Incoming Change**
- **Accept Both Changes**

Välj Person A

Om du väljer: **Accept Current Change**

Julia Åhlén
Lexicon

blir filen: Detta är Person A:s version.

Välj Person B

Om du väljer: **Accept Incoming Change**

blir den: Detta är Person B:s version.

Behåll båda

Du kan också manuellt skriva:

Detta är Person A:s version.

Detta är Person B:s version.

Steg 10 Spara och slutför mergen

När ni har bestämt vilket innehåll som ska vara kvar:

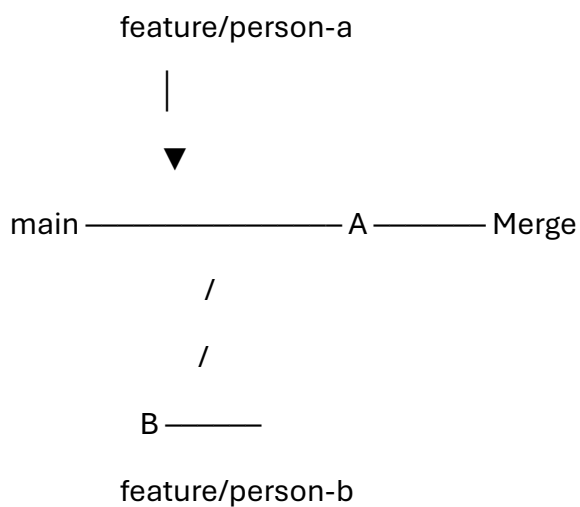
1. Spara test.txt.
2. Gå tillbaka till GitHub Desktop.
3. Konflikten ska nu vara markerad som löst.
4. Klicka **Commit merge**.

Sedan:

Push origin

Resultatet

Git-historiken ser ungefär ut så här:



Och main innehåller nu den version ni valde när konflikten löstes.

DEL 7 Nu ska gruppen bygga ett litet JavaScript-projekt tillsammans.

Projekt

Ni ska skapa en enkel:

To-Do App

Projektet ska innehålla:

index.html

style.css

script.js

Applikationen ska kunna:

- lägga till en uppgift
- visa uppgifter
- markera en uppgift som klar
- ta bort en uppgift
- ha en enkel CSS-design

DEL 8 Skapa Issues på GitHub

Innan ni börjar koda ska gruppen dela upp projektet i olika uppgifter. Dessa uppgifter ska skapas som Issues på GitHub. En Issue beskriver vad som ska göras.

Exempel:

Issue #1 Skapa HTML-strukturen

Skapa inputfält, knapp och lista där uppgifterna ska visas.

Så skapar ni en Issue

Gå till gruppens repository på GitHub.

Steg 1

Klicka på **Issues**

Steg 2

Klicka på **New issue**

Steg 3

Skriv en titel, t ex: Create HTML structure

Steg 4

Skriv vad som ska göras, t ex:

Skapa HTML-strukturen för To-Do Appen.

Appen ska innehålla:

- ett inputfält
- en knapp för att lägga till uppgift
- en lista där uppgifterna ska visas

Steg 5 Tilldela studenten

Under **Assignees** väljer ni den student som ska ansvara för uppgiften, t ex Assignee:

Student 1

Steg 6

Klicka **Create issue**

GitHub ger då Issue:n ett nummer:

#1 Create HTML structure

Fortsätt skapa projektets Issues

Skapa följande Issues tillsammans. Notera att ni kan skapa fler eller färre issues beroende på antalet medlemmar i gruppen.

Issue #1 HTML (redan klar i föregående steg)

Titel: Create HTML structure

Uppgift:

Skapa:

- inputfält
- knapp
- lista för uppgifter

Ansvarig: Student 1

Issue #2 CSS

Titel: Add CSS design

Uppgift:

Skapa:

- layout
- färger
- styling av knappar
- styling av färdiga uppgifter

Ansvarig: Student 2

Issue #3 Lägg till uppgift

Titel: Add task functionality

Uppgift:

Använd JavaScript för att göra det möjligt att lägga till nya uppgifter.

Ansvarig: Student 3

Issue #4 Markera och ta bort

Titel: Complete and delete tasks

Uppgift:

Använd JavaScript för att:

- markera en uppgift som klar

Julia Åhlén
Lexicon

- ta bort en uppgift

Ansvarig: Student 4

DEL 9 Fördela och skapa branches

Nu ska varje student skapa en branch för sin Issue.

Student	Issue	Branch
Student 1	HTML	feature/task-input
Student 2	CSS	feature/design
Student 3	Add task	feature/add-task
Student 4	Complete/Delete	feature/task-actions

Branchens namn beskriver alltså vad studenten arbetar med. Innan du skapar branchen ska du kontrollera att du har den senaste versionen av main.

Öppna terminalen i VS Code.

Kör:

git switch main

Sedan:

git pull

Nu ska du ha den senaste versionen av projektet.

Skapa sedan din branch, t ex för Student 3

git switch -c feature/add-task

Kontrollera:

git branch

Du ska se:

main

* feature/add-task

* visar vilken branch du arbetar på.

DEL 10 Testa din kod

Innan du gör commit ska du testa funktionen.

Exempel för Student 3:

1. Starta projektet.
2. Skriv en uppgift.
3. Klicka på knappen.
4. Kontrollera att uppgiften visas.

Om något inte fungerar:

Julia Åhlén
Lexicon

Fixa → spara → testa igen.

DEL 11 Stage dina ändringar

När du är nöjd med ändringarna ska du göra staging andtingen via Source Control eller i Terminalen.

Kontrollera med:
`git status`

DEL 12 Commit

När ändringarna är staged gör du en commit. I terminalen:
`git commit -m "Add task creation functionality"`

DEL 13 Push till GitHub

Nu ska du skicka din branch till GitHub. Första gången du pushar branchen:

`git push -u origin feature/add-task`

Byt naturligtvis ut branch-namnet mot ditt eget. Efter den första pushen räcker:

`git push`

Nu finns din branch på GitHub.

DEL 14 Koppla Pull Request till din Issue

Nu ska du skapa en Pull Request.

På GitHub går du till:

Pull requests → New pull request

Välj:

base: main

compare: feature/add-task

Det betyder:

feature/add-task

↓

main

Skriv en beskrivning, t ex jag har lagt till funktionalitet för att skapa nya tasks.

Koppla sedan Pull Requesten till din Issue. Skriv exempelvis längst ner:

Closes #3, om din Issue är nummer 3. Det betyder: När Pull Requesten mergas ska GitHub stänga Issue #3. Nu har vi hela kopplingen:

Issue #3

Add task functionality

↓

feature/add-task

↓

kod

Julia Åhlén
Lexicon

↓

commit

↓

push

↓

Pull Request

↓

Closes #3

DEL 15 Code Review

Nu ska en annan student granska din Pull Request.

Studenten ska kontrollera:

- Fungerar koden?
- Uppfyller den Issue:n?
- Är koden begriplig?
- Har funktionen testats?
- Har studenten bara ändrat det som behövs?

Den som granskar kan skriva en kommentar, t ex funktionen fungerar bra. Jag godkänner Pull Requesten. Eller: funktionen fungerar, men knappen behöver också få en CSS-klass.

DEL 16 Merge Pull Request

När Pull Requesten är godkänd, klicka:

Merge pull request

och sedan:

Confirm merge

Nu finns ändringarna i:

main

och den kopplade Issue:n kan stängas automatiskt om ni använt:

Closes #3

DEL 17 Hämta den nya versionen

När en annan students Pull Request har mergats måste du uppdatera din lokala version.

I Terminalen, gå först till:

git switch main

Hämta sedan den nya versionen:

git pull

Nu finns den andra studentens ändringar på din dator. När du ska börja med en ny funktion ska du alltid börja från den senaste main.

git switch main

git pull

Julia Åhlén
Lexicon

Sedan skapar du en ny branch:
git switch -c feature/ny-funktion

DEL 18 Gruppens slutresultat

När alla fyra (eller fler/färre) Issues är färdiga ska main innehålla en fungerande To-Do App. Alla eventuella konflikter skall vara lösta.

T ex:

TO-DO APP

[Skriv uppgift...] [Add]

☐ *Köpa kaffe [Delete]*

☒ *Göra GitHub [Delete]*

☐ *Läsa JavaScript [Delete]*

Och på GitHub ska ni kunna visa:

- 4 Issues eller det antalet som är relevant för er grupp
- 4 feature branches
- commits
- 4 Pull Requests
- code reviews
- merge till main

Ni ska kunna arbeta tillsammans med JavaScript på ett sätt som liknar ett riktigt utvecklingsteam:

Planera → Issue → Branch → Koda → Testa → Commit → Push → Pull Request → Review → Merge → Pull.